

SILLON — Guide d'installation administrateur

Ce guide décrit l'installation de SILLON sur un serveur cible, à destination des administrateurs de la direction. Il complète le [SILLON cahier des charges.md](#) (choix fonctionnels et techniques) et le [README.md](#) (vue d'ensemble, état du projet).

1. Prérequis

1.1 Système cible

- **Debian 13 (Trixie)**, architecture amd64.
- Accès réseau **limité aux dépôts Debian via le proxy d'entreprise** : aucun accès à internet en général, ni à un registre de conteneurs (Docker Hub ou équivalent). Cette contrainte est structurante — voir §4.
- Compte d'installation avec droits `root` (ou `sudo`).

1.2 Réseau

- Le serveur doit être joignable en HTTPS (443) depuis les postes des agents. Le port 80 est utilisé uniquement pour la redirection vers 443.
- Aucun port applicatif (API PostgREST sur 3000, orchestrateur sur 5000, PostgreSQL sur 5432) n'est exposé au-delà de `localhost` : tout le trafic externe transite par Nginx.

1.3 Paquets Debian requis

Les dépendances sont déclarées dans chaque paquet SILLON et installées automatiquement par `apt / dpkg` si le dépôt Debian est accessible : `postgresql-17`, `nginx`, `openssl`, `sudo`, `fail2ban`, `python3-flask`, `python3-psycopg2`, `python3-jwt`, `unicorn`, `podman`. Aucune action manuelle requise en amont.

2. Vue d'ensemble des paquets

Paquet	Rôle	Dépend de
<code>sillon-server</code>	Socle : PostgreSQL, Nginx, API de requêtage (PostgREST), catalogue applicatif, anti-bruteforce	—
<code>sillon-orchestrateur</code>	Création/suppression des bases, SQL libre, import CSV, partage, file d'attente	<code>sillon-server</code>

Paquet	Rôle	Dépend de
<code>sillon-worker</code>	Consommation de la file, lancement des conteneurs d'exécution de scripts	<code>sillon-orchestrateur</code>
<code>sillon-image-execution</code>	Image figée (Python/R) utilisée pour l'exécution des scripts	<code>sillon-worker</code>

L'ordre d'installation ci-dessus est obligatoire : chaque paquet vérifie au `postinst` que le précédent est déjà configuré (présence de `/etc/sillon/secrets.env`, service actif) et refuse de s'installer sinon.

3. Obtention des paquets

Deux cas de figure :

- **Paquets déjà construits** : le dossier `build/` du dépôt contient les `.deb` prêts à l'emploi (`sillon-server_0.1.0_amd64.deb`, `sillon-orchestrateur_0.1.0_all.deb`, `sillon-worker_0.1.0_all.deb`, `sillon-image-execution_0.1.0_amd64.deb`). C'est le cas le plus courant : passer directement à l'étape 4.
- **Reconstruction nécessaire** (nouvelle version, modification du code applicatif) : depuis une machine **ayant accès à internet** (le binaire PostgREST et l'image d'exécution sont téléchargés/construits à cette étape, pas sur la cible) :

```
bash cd build/ ./build.sh
```

Le script vendorise le binaire PostgREST officiel (vérifié par somme SHA256) et construit l'image Podman d'exécution, avant de produire les quatre `.deb`. Voir cahier des charges §12.2 à §12.5 pour le détail.

Transférer ensuite les quatre `.deb` vers le serveur cible (SCP, dépôt APT interne, ou tout autre moyen).

4. Pourquoi cette contrainte de vendoring ?

Point important à comprendre avant d'installer : le serveur cible **n'a jamais accès à un registre de conteneurs ni à internet**, seulement aux dépôts Debian via le proxy d'entreprise. Un `podman build` ou un téléchargement direct lancés au moment de l'installation échoueraient systématiquement (constaté lors des premiers tests de déploiement réels). C'est pourquoi :

- le binaire PostgREST est embarqué tel quel dans `sillon-server` (`/usr/lib/sillon/postgrest`) ;
- l'image d'exécution est construite en amont (hors cible) et embarquée en archive dans `sillon-image-execution` (`/usr/lib/sillon/image-execution.tar`), chargée localement

via `podman load` au `postinst` — jamais reconstruite sur place.

Aucune action requise de l'administrateur sur ce point : c'est purement informatif, pour comprendre pourquoi une tentative de reconstruction sur la cible échouerait.

5. Installation

Sur le serveur cible, dans l'ordre :

```
sudo dpkg -i sillon-server_0.1.0_amd64.deb
sudo apt-get install -f      # résout les dépendances manquantes si besoin

sudo dpkg -i sillon-orchestrateur_0.1.0_all.deb
sudo dpkg -i sillon-worker_0.1.0_all.deb
sudo dpkg -i sillon-image-execution_0.1.0_amd64.deb
```

Ce que fait chaque `postinst` , en résumé

- **sillon-server** : génère les secrets applicatifs (`/etc/sillon/secrets.env` , `chmod 600 root:root`), crée la base `sillon_catalog` et y déploie le catalogue (`schema.sql`), configure l'API PostgREST (`/etc/sillon-api.conf`), génère un certificat TLS auto-signé si aucun n'est présent (`/etc/ssl/sillon/`), active le site Nginx, configure la limitation de débit et Fail2Ban sur `/api/rpc/login` , met en place la purge automatique du journal d'audit, et démarre les services `sillon-api` et `nginx` .

À la première installation, l'identifiant et le mot de passe administrateur initiaux s'affichent dans la sortie du `postinst` : `=== Identifiant administrateur initial : admin@sillon.local ===`
Mot de passe administrateur initial : <génééré aléatoirement> `===` **Noter ce mot de passe immédiatement** : il n'est affiché qu'une seule fois et n'est stocké nulle part en clair.

- **sillon-orchestrateur** : applique les droits sur `/opt/sillon-orchestrateur` , active et démarre le service `sillon-orchestrateur` (Gunicorn, port 5000 en local).
- **sillon-worker** : configure Podman en mode rootless pour l'utilisateur système `www-data` (répertoire `$HOME` dédié, gestionnaire de cgroups `cgroupfs` , plage subuid/subgid, lingering systemd), autorise la connexion PostgreSQL depuis les conteneurs de script (`pg_hba.conf`), puis active et démarre le service `sillon-worker` .
- **sillon-image-execution** : charge l'image d'exécution vendorisée dans le stockage Podman rootless de `www-data` .

Une mise à jour ultérieure (paquets déjà installés)

Les `postinst` détectent une installation existante (présence de la base `sillon_catalog`) et n'effectuent alors que des migrations incrémentales — **aucune recreation destructive**, secrets et

certificat TLS institutionnel conservés tels quels.

6. Vérifications post-installation

```
# Services actifs
systemctl status sillon-api sillon-orchestrateur sillon-worker nginx postgresql
fail2ban

# Journaux en cas de problème
journalctl -u sillon-api -n 50
journalctl -u sillon-orchestrateur -n 50
journalctl -u sillon-worker -n 50
```

Puis, depuis un navigateur : se connecter à `https://<adresse-du-serveur>/` avec l'identifiant administrateur initial noté à l'étape 5, et changer immédiatement ce mot de passe depuis le panneau d'administration.

Le certificat TLS généré par défaut est **auto-signé** (donc affiché comme non fiable par le navigateur). Le remplacer par un certificat institutionnel avant toute mise en production réelle :

```
sudo cp mon-certificat.crt /etc/ssl/sillon/sillon.crt
sudo cp ma-cle.key /etc/ssl/sillon/sillon.key
sudo chown root:root /etc/ssl/sillon/sillon.crt
sudo chown root:www-data /etc/ssl/sillon/sillon.key
sudo chmod 644 /etc/ssl/sillon/sillon.crt
sudo chmod 640 /etc/ssl/sillon/sillon.key
sudo systemctl reload nginx
```

(Un `postinst` ultérieur ne régénère jamais ce certificat s'il est déjà présent — le remplacement ci-dessus est donc définitif jusqu'à intervention manuelle.)

7. Configuration optionnelle

7.1 Notifications par mail

Par défaut, l'orchestrateur tente d'envoyer les notifications de fin de traitement via un relais SMTP local (`localhost:25`), sans authentification. Pour utiliser un relais SMTP de la direction, ajouter les variables suivantes au service (`systemctl edit sillon-orchestrateur` , section `[Service]`) :

```
Environment=SILLON_SMTP_HOST=smtp.exemple-direction.fr
Environment=SILLON_SMTP_PORT=25
Environment=SILLON_SMTP_FROM=sillon@exemple-direction.fr
Environment=SILLON_URL=https://sillon.exemple-direction.fr
```

Puis `systemctl restart sillon-orchestrateur`. Un échec d'envoi de notification est journalisé (`journalctl -u sillon-orchestrateur`) mais ne bloque jamais le traitement lui-même.

7.2 Quotas et paramètres applicatifs

Les quotas (taille maximale d'import CSV, durée maximale d'un job, CPU/RAM/PID par conteneur, quota disque par base, etc.) sont stockés dans la table `parametres` du catalogue applicatif, modifiable sans redémarrage de service ni reconstruction de paquet — soit depuis le panneau d'administration de l'application, soit directement en SQL :

```
UPDATE public.parametres SET valeur = '4096' WHERE cle = 'taille_max_csv_mo';
```

Valeurs par défaut à l'installation :

Paramètre	Valeur par défaut
<code>taille_max_csv_mo</code>	2048
<code>seuil_import_synchrone_mo</code>	10
<code>duree_max_job_minutes</code>	30
<code>cpu_max_conteneur_vcpu</code>	2
<code>ram_max_conteneur_mo</code>	4096
<code>jobs_simultanes_par_utilisateur</code>	1
<code>quota_disque_base_mo</code>	20480
<code>work_mem_default_mo</code>	64
<code>connexions_max_par_utilisateur</code>	5

8. Désinstallation

```
sudo apt-get remove sillon-image-execution sillon-worker sillon-orchestrateur  
sillon-server
```

Les `prerm` / `postrm` ne suppriment jamais la base `sillon_catalog` ni `/etc/sillon/secrets.env` automatiquement (protection contre une perte de données accidentelle). Pour une suppression complète, après désinstallation des paquets :

```
sudo -u postgres psql -c "DROP DATABASE sillon_catalog;"  
sudo rm -rf /etc/sillon /var/lib/sillon /etc/ssl/sillon
```

Rappel : aucun mécanisme de sauvegarde n'est porté par l'application — la protection contre la perte de données relève de l'infrastructure d'hébergement de la direction.

9. Dépannage courant

Symptôme	Cause probable	Vérification
Import CSV volumineux rejeté (413)	Anciennement un défaut Nginx (<code>client_max_body_size</code>) — corrigé, la limite Nginx est illimitée et seule la limite applicative (<code>taille_max_csv_mo</code>) s'applique	<code>grep client_max_body_size /etc/nginx/sites-available/sillon</code>
Script utilisateur ne démarre jamais	Session D-Bus de <code>www-data</code> non encore prête après un redémarrage du serveur	<code>systemctl status user@33.service</code> , puis <code>systemctl restart sillon-worker</code>
<code>sillon-orchestrateur</code> refuse de s'installer	<code>sillon-server</code> non installé/configuré au préalable (<code>/etc/sillon/secrets.env</code> absent)	Installer/vérifier <code>sillon-server</code> d'abord
Script ne peut pas joindre PostgreSQL	Adresse hôte détectée par le <code>postinst</code> de <code>sillon-worker</code> obsolète (changement d'interface réseau)	Réinstaller <code>sillon-worker</code> , ou ajouter manuellement l'entrée dans <code>pg_hba.conf</code>